PROJECT AND TEAM INFORMATION

Project Title

COLLEGE EVENT REGISTRATION PLATFORM

Student / Team Information

Team Name: Team *ID:*	*BRAWL CODEs* *DSCPP-III-2026-T188*
Team member 1 (Team Lead) *Roll No: 2028783* *Section ML3* *Student ID: 2510370169*	*Singhal,Nikunj– 2510370169* *nikunjsinghal726@gmail.com* *2510370169@geu.ac.in* 
Team member 2 *Roll No: 2028724* *Section ML4* *Student ID: 2510011536*	*Kashyap, Harshit – 2510011536* *hk9780031@gmail.com* *2510011536@geu.ac.in* 
Team member 3 *Roll No: 2028688* *Section ML2* *Student ID: 2510371598*	*Istwal, Bhargav – 2510371598* *bhargavistwal5@gmail.com* *2510371598@geu.ac.in* 

Mentor: Dr. Prabhdeep Singh

Email: prabhdeepsingh.cse@geu.ac.in

PROPOSAL DESCRIPTION

Every semester, college campuses run a whole mix of events – technical competitions, workshops, seminars, cultural fests, sports meets, hackathons, and countless other student activities. Yet despite how frequent these events are, registration is still mostly handled the old-fashioned way: paper forms, scattered spreadsheets, or a different online tool for every single event. For students, this quickly turns into a hassle. They have to hunt around just to find out what events are even happening, there's no easy way to check how many seats are left, deadlines get missed, and once they've registered, there's rarely a simple way to track their own registrations in one place.

It's not much easier on the other side either. Event organizers end up juggling participant lists manually, struggling to keep capacity numbers accurate, and spending unnecessary time and effort putting together reports after an event wraps up. The College Event Registration Platform is being built to fix exactly this problem – by bringing everything into one centralized, well-organized system. Students would be able to browse upcoming events, search and filter based on what interests them, look at full event details, register, cancel if their plans change, and always know exactly where their registration stands.

On top of that, the platform would give admins and event organizers the tools they need to run things smoothly – creating, updating, and deleting events, managing seat capacity, viewing who's registered, searching through registrations, and pulling together simple reports.

Beyond solving a genuine, everyday campus problem, this project is also a chance to put Object-Oriented Programming and Data Structures in C++ to real, practical use. Ideas like classes and objects, encapsulation, inheritance, polymorphism, linked lists, queues, stacks, and searching/sorting algorithms won't just stay theoretical here – they'll actually shape how the system works.

State of the Art / Current solution

Right now, most colleges get by using a patchwork of Google Forms, spreadsheets, notice boards, college websites, and the occasional standalone event-management platform. These tools work, more or less, but they were never really designed to talk to each other. As a result, event details, participant lists, and registration status end up scattered across different places instead of living in one system anyone can rely on.

There are, of course, full-fledged online event-management platforms out there that handle things like user authentication, payment processing, email notifications, backend databases, and polished web dashboards. But realistically, building something at that scale isn't the point here – it's far more complex than what a basic academic C++ project needs to cover.

That's why the College Event Registration Platform takes a much simpler, education-focused route instead. Rather than relying on external services or full databases, the system is meant to show how event creation, searching, registration, cancellation, capacity management, and participant tracking can all be handled directly in C++.

What really sets this project apart isn't the feature list – it's the intent behind it. The focus is on demonstrating C++ OOP concepts along with Data Structures and Algorithms through a real, relatable scenario: registering for college events.

Project Goals and Milestones

The main goal here is to build a C++-based system that makes organizing college events and registering for them a whole lot simpler and more organized.

The key goals of the project are to:

1. *Design an object-oriented college event registration system using C++.*
2. *Represent students, events, organizers, and registrations through well-structured classes.*
3. *Allow events to be created, updated, and deleted with ease.*
4. *Let students search, filter, and view details of upcoming events.*
5. *Handle student registration and cancellation while keeping event capacity in check.*
6. *Manage registered students and registration status using the right data structures for the job.*
7. *Add searching and sorting functionality for both events and student registrations.*
8. *Keep the interface simple, clean, and console-based so it stays easy to use.*

Initial Milestones

- *Milestone 1 – Research and Requirement Analysis: Look into how event registration typically works on campus today, and pin down exactly what students, event organizers, and admins would need from the system.*
- *Milestone 2 – System Design: Work out the overall system architecture – class structure, how data structures map to different parts of the system, and how users will move through the menus.*
- *Milestone 3 – Core Implementation: Build out the Student, Event, Organizer, and Registration classes, along with the data structures they'll depend on.*
- *Milestone 4 – Event and Registration Management: Get event creation, updating, deletion, searching, and filtering working, along with the registration and cancellation flow. • Milestone 5 – Data Structure and Algorithm Integration: Bring in the appropriate searching, sorting, and datamanagement algorithms and weave them into the system.*
- *Milestone 6 – Testing and Documentation: Test both normal use cases and edge cases, fix whatever breaks, and pull together the documentation, diagrams, and presentation material ahead of the final demo.*

Project Approach

The project will be built as a modular, console-based C++ application. The first step is to lay out the system design by identifying the core entities involved in college event registration – namely Student, Event, Organizer, and Registration.

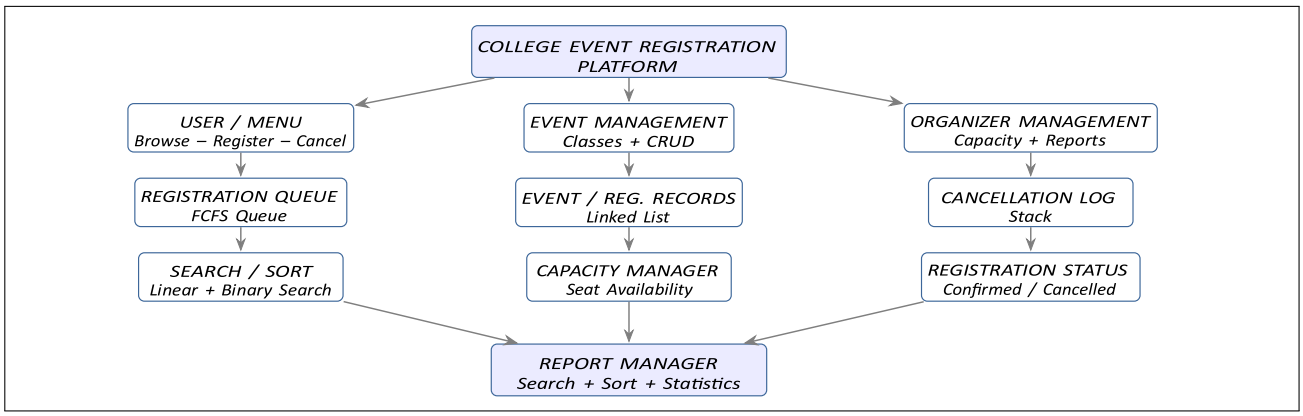
Object-Oriented Programming will form the backbone of how these entities are organized into classes. Encapsulation will be used throughout to keep each class's data members protected and properly managed. Where it makes sense, inheritance and polymorphism will come into play too – for instance, to represent different types of users like students versus administrators or organizers, each with their own behavior but sharing a common structure.

The choice of data structures will depend on what each part of the system actually needs. Linked lists are a natural fit for maintaining dynamic collections of events or registrations, since these can grow or shrink as events are added or students sign up. Queues will be used wherever registrations need to follow a strict first-come-first-served order. Stacks will come in handy for keeping track of cancellations or recent operations, giving the system a simple way to look back at what's just happened. Searching and sorting will also play a key role in keeping things efficient. Linear or binary search can be used to look up event records quickly, while sorting algorithms will help arrange events by date, name, or number of available seats – whatever makes sense for the user browsing them.

Whenever a student tries to register, the system will first check the event's remaining capacity. If the event is already full, the registration simply won't go through. Cancellations, on the other hand, will update the registration records accordingly and free up the seat again for someone else.

On the technical side, the project will be developed in C++ using a standard compiler, with an IDE like Visual Studio Code or Visual Studio for development. Git/GitHub will likely be used for version control and to support collaborative work. Rather than building everything at once, the system will be developed module by module, with each part tested individually before everything is finally integrated together.

System Architecture (High Level Diagram



Project Outcome / Deliverables

By the end of this project, the goal is to have a fully functional, C++ console-based College Event Registration Platform capable of managing college events and student registrations from start to finish. The key deliverables include:

- *A working College Event Registration Platform, built entirely in C++.*
- *Object-oriented classes representing students, events, organizers, and registrations.*
- *Support for creating, modifying, and deleting events.*
- *Event searching and filtering functionality.*
- *Student registration and cancellation functionality.*
- *Proper management of event capacity and seat availability.*
- *Use of suitable data structures such as linked lists, queues, and stacks.*
- *Searching and sorting algorithms applied to event and registration records.*
- *Registration status tracking, so students always know where they stand.*
- *Basic reports covering event-wise and registration-related data.*
- *A simple, easy-to-navigate console-based interface.*
- *UML/class diagrams and system architecture diagrams.*
- *Test cases along with their results.*
- *Complete PBL documentation and presentation material.*
- *A final demonstration walking through the full process – from browsing an event, to registering, cancelling, and generating reports.*

Ultimately, the finished system should show how C++ Object-Oriented Programming, Data Structures, and Algorithms can come together in a practical, real-world way to solve a genuine college event-management problem.

Assumptions

The project assumes the college already has a defined set of students, events, and event organizers to work with. Student and event information will either be entered manually or loaded in as predefined sample data. Each event is assumed to have a fixed maximum capacity, and registrations will only be accepted as long as seats remain available. Every student is assumed to have a unique ID, and the system will not allow a student to register for the same event twice. This project is meant purely as an educational simulation and does not rely on real college servers, online payment systems, email services, or external authentication – all event and registration data used for testing will simply be simulated.

References

1. *cppreference – C++ Classes and Objects: https://en.cppreference.com/w/cpp/language/classes.html*
2. *cppreference – C++ Inheritance: https://en.cppreference.com/w/cpp/language/derived_class.html*
3. *cppreference – C++ Polymorphism: https://en.cppreference.com/w/cpp/language/virtual.html*
4. *cppreference – C++ Data Structures and Containers: https://en.cppreference.com/w/cpp/container.html*
5. *cppreference – C++ Algorithms and Searching/Sorting: https://en.cppreference.com/w/cpp/algorithm.html*
6. *GeeksforGeeks – Data Structures and Algorithms: https://www.geeksforgeeks.org/data-structures/*
7. *C++ Documentation / Standard Library Reference: https://en.cppreference.com/w/cpp/standard_library.html*